# Embedded Linux & System Software Interview Handbook

## Chapter 3: U-Boot - The Universal Bootloader

**Goal:** Understand the architecture, responsibilities, configuration, debugging, and interview concepts of U-Boot.

**What is U-Boot?**
U-Boot (Universal Bootloader) is the first fully featured software that executes after BootROM/SPL. It initializes hardware, loads the Linux kernel, Device Tree and initramfs into RAM, and transfers execution to the kernel.

**Boot Flow**
Power On → BootROM → SPL → U-Boot → Linux Kernel → Device Tree → initramfs → RootFS → systemd

**Main Responsibilities**
1. Initialize DDR memory.
2. Configure CPU clocks.
3. Initialize UART for console logs.
4. Detect storage devices (eMMC, NAND, SD).
5. Initialize Ethernet and USB if required.
6. Load kernel, DTB and initramfs into RAM.
7. Pass kernel boot arguments.
8. Jump to the Linux kernel entry point.

**Important Components**
Board configuration, drivers, command shell, environment variables, filesystem support (FAT/EXT4), network stack, TFTP, scripting and secure boot support.

**Environment Variables**
bootcmd, bootargs, bootdelay, ipaddr, serverip, ethaddr. These determine how and from where Linux boots.

**Frequently Used Commands**
printenv, setenv, saveenv, bdinfo, version, md, mw, mm, ext4load, fatload, tftpboot, booti, bootm, reset.

**Typical Boot Sequence**
Initialize hardware → Read environment → Load kernel image → Load Device Tree → Load initramfs (optional) → Execute booti/bootm → Transfer control to Linux.

**Memory Layout**
Kernel, Device Tree and initramfs must be loaded at non-overlapping RAM addresses. U-Boot is responsible for placing these images correctly before booting Linux.

**Debugging Tips**
Use UART to observe logs. Verify printenv values. Ensure bootargs point to the correct root filesystem. Confirm kernel and DTB load addresses. Test storage with ext4ls/fatls. Validate memory using md and mw commands.

**Real Samsung Example**
During Smart TV platform development, UART logs from U-Boot were used to verify DDR initialization, storage detection and successful loading of the kernel. If boot stopped before the Linux banner, engineers focused on U-Boot configuration and memory initialization.

**Common Failure Scenarios**
Incorrect bootargs, corrupted kernel image, invalid Device Tree, wrong RAM load address, DDR initialization failure, missing root filesystem, storage driver issues.

**Interview Questions**

1. What is U-Boot?
2. Why is SPL required?
3. Difference between U-Boot and BIOS/UEFI?
4. What are bootargs?
5. Explain booti vs bootm.
6. How do you debug a board stuck in U-Boot?
7. Why is UART critical during bring-up?
8. How does U-Boot pass information to the Linux kernel?

**Key Takeaways**

U-Boot bridges hardware initialization and Linux startup. A strong understanding of its commands, environment variables, memory layout and debugging techniques is essential for board bring-up and embedded Linux interviews.